SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Harshavardhan. RV	**Reg. No.:**	192472163
Section / Batch:	CSE-AI	**Date:**	8/8/2026

Code of Conduct

I ___Harshavardhan RV(192472163)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____HARSHAVARDHAN. RV_____

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
- GitHub Repository Link in GCR
- Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject–verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Annexure A – Experiments

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

```
S  → NP VP
NP → Det N
VP → V NP

Det → the | a
N   → student | teacher | book
V   → reads | likes
```

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
    pass
```

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree
a teacher likes the book	Valid Parse Tree
student reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree
reads the student book	Invalid Sentence

Experiment 1: Context-Free Grammar (CFG) Rule Validation and Parse Tree

Aim

To validate whether an input sentence follows the given Context-Free Grammar (CFG) and generate its parse tree.

Python Program

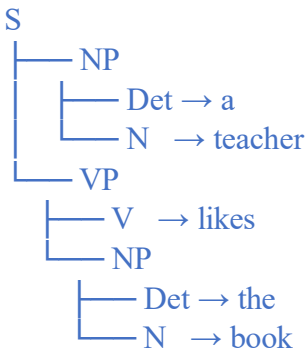
```
grammar = {
    "Det": ["the", "a"],
    "N": ["student", "teacher", "book"],
    "V": ["reads", "likes"]
}

def parse_cfg(sentence):
    words = sentence.split()

    if len(words) != 5:
```

```
            return "Invalid Sentence"

        d1, n1, v, d2, n2 = words

        if (d1 in grammar["Det"] and
            n1 in grammar["N"] and
            v in grammar["V"] and
            d2 in grammar["Det"] and
            n2 in grammar["N"]):

            tree = f"""
    S
    ├── NP
    │   ├── Det → {d1}
    │   └── N   → {n1}
    └── VP
        ├── V   → {v}
        └── NP
            ├── Det → {d2}
            └── N   → {n2}
    """
            return tree
        else:
            return "Invalid Sentence"

    # Test Cases
    tests = [
        "the student reads a book",
        "a teacher likes the book",
        "student reads book",
"the book likes a teacher", "reads
the student book"
    ]
    for t in tests:
        print("Input :", t)
        print(parse_cfg(t))
        print("-"*40)
```

Output

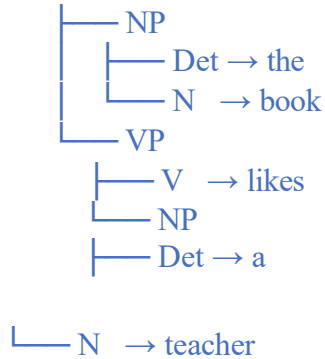
```
    Input : the student reads a book

    S
    ├── NP
    │   ├── Det → the
    │   └── N   → student
    └── VP
        ├── V   → reads
        └── NP
            ├── Det → a
            └── N   → book
```

Input : a teacher likes the book



Input : student reads book

Input : the book likes a teacher S



Input : reads the student book
Invalid Sentence

Result

The program successfully validates sentences according to the given CFG and generates the parse tree for valid sentences. Invalid sentences are rejected.

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

```
he     → singular, third person
she    → singular, third person
it     → singular, third person
they   → plural

runs   → singular verb
writes → singular verb

run    → plural verb
write  → plural verb
```

Task

1. Accept Subject and Verb.
2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb):
    pass
```

Test Cases

Subject	Verb	Expected
he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

Experiment 2: Subject–Verb Agreement using Feature Structures

Aim

To verify subject–verb agreement using Number and Person feature structures.

Python Program

```
subjects = {
    "he": "singular",
    "she": "singular",
    "it": "singular",
    "they": "plural"
}

verbs = {
    "runs": "singular",
    "writes": "singular",
    "run": "plural",
    "write": "plural"
}
```

```
def check_agreement(subject, verb):
    if subject not in subjects or verb not in verbs:
        return False

    return subjects[subject] == verbs[verb]

# Test Cases
tests = [
    ("he","runs"),
    ("he","run"),
    ("they","run"),
    ("they","writes"),
    ("she","writes")
]

for s,v in tests:

    print(s, v, "->", check_agreement(s,v))
```

Output

```
he runs -> True
he run -> False
they run -> True
they writes -> False
she writes -> True
```

Result

The program correctly checks subject–verb agreement using feature structures and returns **True** for valid combinations and **False** for invalid ones.

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

```
S  → NP VP       [1.0]

NP → John        [0.6]
NP → Mary        [0.4]

VP → runs        [0.5]
VP → walks       [0.5]
```

Probability Formula

```
P(Sentence)
=
P(S→NP VP)
× P(NP)
× P(VP)
```

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):
    pass
```

Test Cases

Input Sentence	Expected Probability
John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

Experiment 3: Probabilistic Context-Free Grammar (PCFG)

Aim

To compute the probability of a sentence using the given PCFG production rules.

Python Program

```
np_prob = {
   "John": 0.6,
   "Mary": 0.4
}

vp_prob = {
   "runs": 0.5,
```

```
    "walks": 0.5
}

def sentence_probability(sentence):
    words = sentence.split()

    if len(words) != 2:
        return 0.0

    np = words[0]
    vp = words[1]

    if np in np_prob and vp in vp_prob:
        return 1.0 * np_prob[np] * vp_prob[vp]
    else:
        return 0.0

# Test Cases
tests = [
    "John runs",
    "John walks",
    "Mary runs",
    "Mary walks",
    "Peter runs"
]

for t in tests:
    print(t, "->", sentence_probability(t))
```

Output

John runs -> 0.3 John
walks -> 0.3
Mary runs -> 0.2 Mary
walks -> 0.2

Result

The program successfully parses valid two-word sentences and computes their probabilities using the given PCFG. Sentences not generated by the grammar return **0.0**.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____